

AlgoFlow 软件架构与技术设计

版本：2.1

日期：2026-08-20

架构基线：OpenHarmony 原生手机端 + 浏览器桌面工作台 + 华为云服务

1. 架构目标

AlgoFlow 要解决的不是“多端各做一个笔记应用”，而是让用户在手机和电脑之间连续推进同一个算法思路：捕获自然语言、整理为伪代码或短代码、隐藏并复写、检查可行性、处理冲突并保留演进历史。

架构优先级如下：

1. 不丢失用户思路和代码。
2. AI 忠实转换与提示/解题严格隔离。
3. 手机和电脑可离线编辑并可靠同步。
4. OpenHarmony 服务卡片具备可验证的原生价值。
5. 三人团队能在比赛周期内构建、测试、部署和维护。

2. 关键架构决策

2.0 已验证工程基线

当前工程路径为 C:\study\AlgoFlow，已完成一次 HAP Debug 构建，构建日志结果为 BUILD SUCCESSFUL。已确认配置：

配置	值
DevEco Studio	6.0.0 Release (构建 6.0.0.858)
SDK	OpenHarmony SDK API Version 20
SDK 组件	ArkTS、JS、Previewer、Toolchains 6.0.0.47
工程模型	Stage, ArkTS + ArkUI
设备类型	Phone
Compatible/Target SDK	6.0.0(20) / 6.0.0(20)
runtimeOS	工程模板实际值为 HarmonyOS
签名	未配置 signingConfigs，仅完成未签名 Debug HAP 构建

这里必须区分：SDK 管理器的分发名称是 OpenHarmony SDK，但工程 runtimeOS 当前为 HarmonyOS。服务卡片、分布式能力和上架资格均不能仅凭 API 20 或 SDK 名称推断，必须完成对应 API 编译和设备验证。不得手工改写 runtimeOS。

决策	选择	理由
OpenHarmony 客户端	ArkTS + ArkUI 原生	直接接入服务卡片、权限、生命周期和本地数据
电脑端	响应式 Web 工作台	覆盖常见电脑系统，无需额外安装和多套打包链

决策	选择	理由
跨端复用	共享协议、Schema、规则和测试向量	保留平台体验，降低 UI 适配和社区框架风险
同步	华为云 API + 乐观并发 + 冲突副本	实现可解释、不静默丢数据的首版同步
AI	供应商中立 Gateway + 结构化输出	隔离密钥、模型差异和行为校验
OpenHarmony 特色	服务卡片优先	与“随时捕获和继续”场景直接相关，范围可控
代码执行	MVP 不执行任意代码；P1 受控云沙箱	避免手机安全风险，保留快速验证升级路径

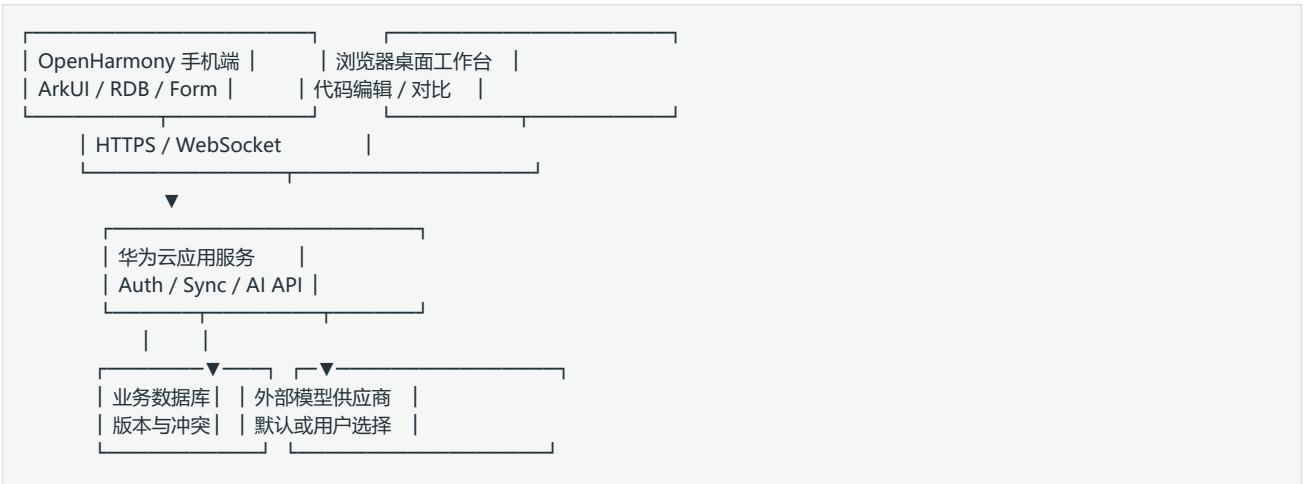
3. 跨平台白皮书的采用边界

《跨平台框架白皮书 V1.0》强调按 UI 层、业务逻辑层和平台能力层进行选型，并明确不存在适用于所有项目的万能框架。本项目采纳这一方法，而不是直接采用某个框架。

方案	当前判断	主要原因
原生 ArkUI + Web	MVP 采用	原生能力清晰，电脑覆盖广，团队工具链较少
CMP/KMP	P2 预研	桌面与 OpenHarmony 共享潜力高，但 OpenHarmony 适配和团队 Kotlin 能力需验证
Flutter	P2 预研	UI 一致性好，但插件、引擎、包体积和原生卡片桥接增加成本
React Native	暂不采用	主要价值偏移动端，不能直接解决浏览器桌面工作台和同步问题
Electron/桌面壳	暂不采用	Web 已能交付核心功能，桌面打包会增加发布维护面

框架升级门槛：必须由可运行原型证明 Web 存在关键不足，同时验证 OpenHarmony 目标版本、系统能力接入、编辑性能、包体积、启动时间、插件覆盖和 CI 成本。框架复用率不能代替这些证据。

4. 系统上下文



普通云同步走应用服务，不依赖同局域网设备发现。OpenHarmony 分布式接续未来作为单独适配器接入，不改变同步核心协议。

5. 逻辑分层



依赖只允许由外向内。领域层不能引用 ArkUI、DOM、数据库驱动、HTTP 客户端或模型 SDK。

6. 客户端设计

6.1 OpenHarmony 手机端

建议模块：

```

openharmony/
├── entry/
│   └── src/main/ets/
│       ├── pages/
│       ├── components/
│       ├── viewmodels/
│       ├── application/
│       ├── domain/
│       ├── repositories/
│       ├── services/
│       ├── db/
│       └── sync/
└── form_widget/ # 仅在实际模板要求独立模块时存在
  
```

主要页面：工作区列表、草稿编辑、AI

模式面板、隐藏/复写、版本对比、冲突解决和设置。代码区聚焦阅读与短编辑，不在首版复制完整桌面 IDE。

本地数据库保存工作副本、离线队列和同步游标。服务卡片读取最小摘要数据，并通过稳定草稿 ID 跳转；卡片不显示完整代码。

6.2 浏览器桌面工作台

桌面端提供：

- 工作区和草稿导航；
- 适合键鼠的代码编辑器；
- 原始思路、AI 结果和用户复写并排/差异查看；
- 同步、离线和冲突状态；
- AI 模式选择及来源映射查看。

首版使用浏览器持久化保存未提交变更和离线操作。编辑器实现可选择成熟 Web 编辑器组件，但引入依赖前检查许可证、包体积、浏览器支持和移动端无关代码是否隔离。

7. 领域模型

7.1 核心实体

实体	关键字段
Workspace	workspace_id, owner_id, sync_cursor, version
Draft	draft_id, title, language, status, version, deleted_at
IdeaSegment	segment_id, draft_id, order, text, version
CodeDocument	document_id, draft_id, origin, content, language, base_draft_version
AIArtifact	artifact_id, mode, payload, model_id, rule_version, source_version
SyncOperation	operation_id, entity_type, entity_id, base_version, payload
ConflictRecord	conflict_id, server_version, client_version, resolution

CodeDocument.origin 至少区分 USER、AI_FAITHFUL、USER_REWRITE、AI_HINT 和 AI_SOLUTION，防止不同来源互相覆盖。

7.2 AI 模式枚举

```
FAITHFUL_TRANSFORM
FEASIBILITY_ANALYSIS
PROGRESSIVE_HINT
FULL_SOLUTION
```

FULL_SOLUTION 只能由显式用户操作触发。所有 AI 产物都是不可变版本；重新生成创建新记录，不原地修改历史。

8. AI 服务设计

8.1 调用链

```
用户输入
→ 拆分并保存 IdeaSegment
→ 选择 AI 模式
→ 客户端输入校验
→ AI Gateway 认证、限流和供应商路由
→ 模型结构化输出
→ Schema 解析与来源映射校验
→ 模式越界检测
→ 合格：保存独立 AIArtifact
→ 不合格：受限重试或返回可解释错误
```

8.2 忠实转换契约

```
{
  "mode": "faithful_transform",
  "pseudocode": [
    {
      "step": "按用户描述执行一个步骤",
      "source_refs": ["idea_segment_1"]
    }
  ],
  "code_snippet": null,
  "assumptions": [],
  "missing_information": [],
  "risk_flags": [],
  "added_algorithm_steps": []
}
```

```
}

```

校验规则：

- `source_refs` 必须引用当前请求中真实存在的片段。
- 忠实模式的 `added_algorithm_steps` 必须为空。
- 代码长度、文件数量、语言和危险内容必须满足限制。
- 模型输出不得直接覆盖用户文本或复写文档。
- 复杂度和风险属于分析性标注，不等同于证明结论。

8.3 供应商适配

```
AIProvider
├── TeamDefaultProvider
├── UserConfiguredProvider
└── DisabledProvider

```

Gateway 统一超时、重试、限流、模型映射和审计

ID。供应商密钥不下发到客户端；用户自带密钥的存储与转发方案在安全评审后确定。

9. 同步架构

9.1 状态机

```
LOCAL_ONLY → QUEUED → SYNCING → SYNCED
      |           |
      └─ FAILED ─┘
      └─ CONFLICT → RESOLVED → QUEUED

```

9.2 推送与拉取

1. 客户端读取本地待提交操作。
2. 每个操作携带唯一 `operation_id` 和 `base_version`。
3. 服务端按 `operation_id` 幂等处理。
4. 版本匹配则提交并返回新版本；不匹配则返回服务器当前版本。
5. 客户端保存 `ConflictRecord` 和本地冲突副本。
6. 推送完成后按游标拉取其他设备变更。
7. 整批应用成功后更新本地同步游标。

删除通过墓碑同步。首版不采用“最后写入者获胜”，因为它会静默丢失手机或电脑端修改；也不引入 CRDT，以控制实现复杂度。

9.3 一致性边界

- 单实体写入由服务端事务保证。
- 跨实体操作使用应用层事务或可重试 workflow。
- AI 生成基于固定 `source_version`；草稿变化后旧结果仍可查看，但标记为“基于旧版本”。

- 服务卡片允许短时间缓存，跳转主应用后以最新数据为准。

10. 华为云部署

```

Internet
→ HTTPS 入口 / WAF (按资源选择)
→ API 服务
  ├── Auth 模块
  ├── Sync 模块
  ├── AI Gateway
  └── P1 Runner Queue
→ 关系数据库
→ 缓存/队列 (仅在实际需要后引入)
  
```

开发环境与发布候选环境分离。配置通过云端密钥或环境配置管理，不提交 GitHub。CI 可在 GitHub 执行通用测试，DevEco 构建和设备测试根据可用工具放在本地或华为云；不要求所有编码在云端进行。

10.1 可选代码沙箱

云端 C++ 执行属于 P1，启用前必须满足：

- 每次任务使用固定、只读基础镜像和一次性工作目录；
- 禁止外网访问和宿主机挂载；
- 限制 CPU、内存、进程数、输出大小和执行时间；
- 编译与运行分阶段记录状态；
- 任务完成或超时后强制回收；
- 对用户、IP 和工作区设置并发与费用配额。

未满足全部条件时只提供手工样例推演和静态可行性分析。

11. OpenHarmony 平台能力

11.1 服务卡片

优先能力：快捷记录、最近草稿、待复写提醒、同步状态和定向跳转。最终选择至少三项，以目标 SDK 和卡片尺寸为准。

验证内容：卡片添加、刷新、离线、未登录、草稿删除、点击跳转、后台限制和数据最小化。验证状态统一为 待预研 / 开发中 / 本地验证 / 设备验证 / 可演示。

11.2 两台云手机

两台独立云手机可以验证：同账号登录、云端同步、离线恢复、冲突和卡片在不同实例上的行为。

它们不必然可以验证：设备发现、软总线、近场分享、应用接续和同账号分布式权限。只有云手机服务明确支持并在两个实例上成功复现，才进入完成状态。

12. 安全、隐私与许可证

- 用户思路、代码和 AI 结果默认按私有内容处理。
- 日志只保留请求 ID、状态、耗时、版本和错误类别。

- 客户端和仓库不保存生产密钥、签名、Token 或真实用户数据。
- 外部模型数据流必须与用户选择和后续隐私说明一致。
- Apache License 2.0 仅约束开源代码许可，不替代隐私政策。
- 正式签名和密钥由明确负责人保管，位置和轮换方式在发布前记录，但不写入公共文档。

13. 测试与验证矩阵

范围	本地	华为云/云手机	完成依据
ArkUI 页面	DevEco Previewer/模拟器	云手机	长文本、输入法、后台恢复通过
服务卡片	模板和局部调试	支持的云手机/设备	刷新、跳转和异常录像
Web 工作台	浏览器测试	测试环境	编辑、离线和同步通过
同步 API	单元/集成测试	部署环境	幂等、冲突、删除和恢复通过
AI 契约	Mock + 40 条测试	实际模型测试	来源映射和模式隔离通过
分布式能力	不作结论	两台支持设备	发现、授权、传输和恢复通过
云代码执行	禁止默认开放	隔离环境	安全门禁和压力测试通过

14. 三人交付方式

- 成员 A：OpenHarmony 应用、服务卡片和手机体验。
- 成员 B：Web 工作台、同步协议和华为云部署。
- 成员 C：AI Gateway、测试体系和发布/赛事材料。

GitHub 使用短分支和 Pull Request。Schema、同步协议、数据库迁移、权限、构建配置和签名流程的变更必须经过另一名成员审查。至少两人能够独立完成 DevEco 构建和云端部署。

15. 演进路线

阶段	架构交付
0：技术验证	锁定 SDK；验证 RDB、服务卡片、Web 编辑器、同步最小链路
1：单端 MVP	手机草稿、忠实转换、隐藏复写、离线队列
2：双端闭环	Web 工作台、账号、乐观并发、冲突副本
3：比赛特色	服务卡片可演示、用户测试、发布候选
4：增强验证	云沙箱、渐进提示、分布式能力或跨平台框架原型

16. 架构风险

风险	处置
三人范围过大	Web 首版、不做桌面壳；服务卡片优先；P2 冻结
AI 改写用户思路	来源映射、模式隔离、越界校验、不可变版本
手机电脑互相覆盖	operation_id、base_version、冲突副本
OpenHarmony 能力不可用	SDK 门禁、云手机验证、候选能力不宣传完成

风险	处置
外部模型不稳定或昂贵	Provider 抽象、限流、禁用状态和用户自带 API 预留
云端执行安全风险	MVP 不开放；P1 通过完整沙箱门禁后启用
发布账号/签名缺失	架构冻结阶段确定负责人和受控保管位置

17. 架构完成定义

只有当公共契约版本化、双端构建通过、同步不丢数据、AI 忠实模式通过来源映射测试、服务卡片在目标环境可重复演示、权限和密钥处理已审查，并且三份项目文档一致时，才能将对应架构能力标记为完成。

参考资料：《跨平台框架白皮书 V1.0》，2026-06-01。采用其分层选型方法，不将其中任何社区适配方案视为本项目已验证能力。